Where to Work
Data Prep Manual.

Practices, Framework and Methods to build Where To Work projects.

wtw-data-prep2

Version: 2026-10-23

Table of Contents

Introduction 3
WTW Project Files 3
Data Prep Prerequisites 3
Hardware Dependencies 3
Software Dependencies 4
Data Dependencies 4
Recommended RStudio Configuration 4
Methodological Framework 5
Preparation Scenarios 5
Data Prep Concepts 5
Data Prep Pipeline 6
Initial setup 6
Data Prep Folder Structure and Run Steps 8
wtw-metadata.csv 11
Removing Data 12
Adding Regional Data 12
ArcGIS custom toolbox (wtw-utils) 13
Script Tools 14
Dataprep.csv 15
Columns and data (dataprep.csv) 15
NAT_DATA Bundle (S: drive) 17
Versioning 17
Updating (copy from S: drive) 17
wtw-data-prep2 17
Updating (`git pull`) 17
Regional GIS Data Management 18

Introduction

Where to Work (WTW) web application is a decision support tool for prioritizing conservation efforts. It provides an interface for systematic conservation planning, leveraging optimization algorithms from the prioritizr package. WTW is hosted on the Sites platform, which includes additional planning tools.

This manual serves as a guide to building WTW projects by walking users through the wtw-data-prep2 repository. The repository contains R and Python code, along with a custom ArcGIS Toolbox to support regional data preparation. It covers shaping spatial data, creating metadata, and running the scripts needed to complete a project from start to finish.

WTW Project Files

WTW projects consist of four files that form a fully customized project. These files are required whether uploading via the "upload project data" method or using a built-in project on the server. The difference is convenience; built-in projects are pre-copied to the server for direct access. In both cases, the files are optimized for compression and smaller size, making them easier to upload and share.

- `configuration.yaml`
  The configuration yaml file defines project attributes, data legend elements and map display in the left sidebar "Table of contents" and initial goals in the "New solution" right sidebar.

- `spatial.tif`
  The TIFF file defines the spatial properties of the planning units, such as cell size, extent, number of rows, number of columns and coordinate reference system. It acts as the template to build rasters from columns within the `attribute.csv.gz`.

- **`attribute.csv.gz`**
  The attribute file (zipped `.csv`) defines the cell values of each theme, weight, `include` and exclude in tabular form. Each column in the `.csv` stores the cell values of a single raster variable.

- `boundary.csv.gz`
  The boundary file (zipped `.csv`) defines the adjacency table of each theme, weight, include and exclude. It stores information on the perimeter and shared boundary lengths of the planning units. This is needed to run optimizations for spatial clustering.

Data Prep Prerequisites

All data processing is done on local machines to reduce the load on AVD. This requires installing and configuring the necessary software dependencies, as well as storing local copies of spatial data to be accessed by the scripts.

Hardware Dependencies

- Windows 11.

- It is recommended to have at least 16 GB of RAM.
- It is recommended to have adequate local storage. Required space varies based on the number of active projects, the size of each WTW project, and the number of spatial layers included.

Software Dependencies

Some of these dependencies require administrative privileges to set up. Since most NCC laptops don't provide users with admin rights, you'll need to submit an IT ticket for installation and configuration.

- **R version 4.4.1**: The R version currently used to run the WTW web application. It is recommended to install R directly on the C: drive (`C:\R` or `C:\Program Files\R`).
- **RStudio**: Recommended IDE for running and editing R scripts.
- **Rtools 4.4**: Required for building and installing R packages from source. It is recommended to install Rtools directly on the C: drive (`C:\rtools44`).
- **ArcGIS Pro 3.x**: Used for displaying layers and running custom ArcGIS script tools. The toolbox is tested with Pro 3.7.1. The **01b Extract Raster to Polygon** tool also needs the exactextract Python package, installed once per user in a cloned ArcGIS Pro Python environment, or in the Python user folder where disk space is limited (see Initial setup).
- **Git**: For version control and managing code repositories.
- **Personal GitHub account**: Needed to clone and sync IT managed GitHub repos locally.

Data Dependencies

- **NAT_DATA bundle:** national datasets and metadata located on S: drive.
- **Regional GIS layers:** conversion-ready vector/raster datasets.
- **Area of interest shapefile polygon:** the area defining the where to work project.

Recommended RStudio Configuration

- Pin RStudio to your taskbar for quick access
- Under Tools -> Global Options, "General":
    - Uncheck "Restore most recently open project at startup"
    - Uncheck "Restore `.RData` into workspace at startup"
    - Set "Save workspace to `.RData` on exit" to "Never"
    - Uncheck "Always save history (even when not saving `.RData`)"
    - Uncheck "Remove duplicate entries in history"
- Under Tools -> Global Options, "Appearance":
    - Set the Editor theme to your preference (e.g. solarized dark)
    - Customize as needed

Methodological Framework

Preparation Scenarios

The following outlines three preparation scenarios for building a WTW project. This includes a National 1km Grid, Custom Grid, or Polygon-based projects.

- **NAT 1KM:** national 1km raster grid project using curated Canada-wide datasets from NAT_DATA (stored on S: drive). These production layers have been harmonized to the *Canada_Albers_WGS_1984* projection and aligned with NCC's national 1km grid. This approach is supported by a complete end-to-end automation pipeline that can process projects in minutes. Regional datasets can also be incorporated using wtw-utils to align layers with the national 1 km grid.

- **CUSTOM GRID**: custom raster grid built to match specific regional needs (e.g. 10 ha planning units). This approach requires additional preparation by the analyst. Support tools in wtw-utils can assist with custom grid creation and aligning data to planning units.

- **POLYGON:** irregular vector polygons to define planning units (e.g. watersheds, parcel boundaries). This approach requires additional preparation by the analyst. Support tools in wtw-utils can assist with aligning data to planning units.

Data Prep Concepts

The following outlines the conceptual steps for building a WTW project. These include defining your planning units and projection, extracting spatial data to those planning units, creating the `wtw-metadata.csv`, and running the WTW formatting script.

1. **Define your planning units and projection**
   Planning units are areas that can be managed independently from one another. From a spatial perspective, they are discrete boundaries that can take the form of either:

   - Regular raster grid, GeoTIFF (equal-area, square cells), or
   - Irregular polygons, Shapefile (e.g., watersheds, parcel boundaries, ecoregions).

   You must select a single format for your project. Where To Work is optimized for raster grid projects, which are recommended for large geographic areas, as the WTW application can efficiently handle up to 700,000 planning units.

   Irregular polygon projects are only suited for analysis with fewer than 50,000 planning units, due to display performance limitations.

2. **Extract spatial data to planning units**
   How you extract data depends both on the planning unit format you choose and the type of spatial data you're working with, such as vector points, lines, polygons, or raster data that is either categorical or continuous. The most common metrics that are extracted to planning units include:

- **Area**: e.g., overlap of species range maps, habitat, existing conservation, land cover
- **Length**: e.g., overlap of shorelines, rivers, roads
- **Count**: e.g., number of observations or point features within a planning unit
- **Zonal statistics**: e.g., mean, min, max, etc. values from continuous surfaces like climate or elevation data

3. **Build** `wtw-metadata.csv`
   The `wtw-metadata.csv` file records information for each conservation variable. It is a required structured input for formatting spatial data into a WTW project.

4. **Run the Where to Work formatting script**
   Using the prepared spatial data and completed `wtw-metadata.csv`, run the WTW formatting script. This will output the four standardized input files required to upload your project to the WTW on Sites.

Data Prep Pipeline

The following outlines the required steps for running a **National 1km WTW project**, which is the recommended first preparation scenario for any WTW project. This process involves organizing local files and data, creating and updating metadata sheets, running the necessary scripts, and validating the outputs.

Initial setup

Initial setup is required to copy and extract the **NAT_DATA** bundle from the S: drive to your local machine, organize the folder structure, and clone the **wtw-data-prep2** repository, and set up the ArcGIS Pro Python environment used by the wtw-utils toolbox.

1. **Copy and extract NAT_DATA**
   NAT_DATA houses Canada-wide production datasets that have been harmonized to the *Canada_Albers_WGS_1984* projection and aligned with NCC's national 1 km grid. The data is structured to support compatibility with processing pipelines and applications.

   Each version of the bundle is archived as a zipped folder suffixed with its creation date in YYYYMMDD format. Users should copy the most recent version to their local machine. When updates are made to these datasets, WTW prep users will be notified to refresh their local copies.

   - NAT_DATA is zipped and stored on the S: drive:
     `S:/CONS_TECH/PRZ/DATA/NAT_DATA/__VERSIONS__`

   - Copy the latest zipped folder off the S: drive into a local directory, e.g.
     `C:/PRZ/NAT_DATA/__VERSIONS__`

- Extract the folder one level up so that the unzipped data sits alongside it, e.g. `C:/PRZ/NAT_DATA/NAT_DATA_20250801`

2. **Clone wtw-data-prep2 GitHub repository:**
   https://github.com/NCC-CNC/wtw-data-prep2
   The wtw-data-prep2 repository contains the R and Python code needed to take a project from start to finish, along with a custom ArcGIS Toolbox to support regional data preparation. Follow these steps to clone the repository onto your local machine:
   - Create a directory to hold repositories directly on your C: drive e.g. `C:/Github`.
   - Open **RStudio** and confirm **R version 4.4.1** is displayed in the session window.
   - Create a new version control project using RStudio:
     In RStudio, go to **File** → **New Project** → **Version Control** → **Git**
     - In the **Repository URL** field, paste the URL for the wtw-data-prep2 repository:
       You can find this URL on GitHub by clicking the green **Code** button and selecting the **HTTPS** tab to copy the web URL.
     - In the **Project directory** name, use wtw-data-prep2.
     - In the **Create project as subdirectory of**: Browse to your new repository directory
   - Click Create Project:
     This will clone all the files into a wtw-data-prep2 folder nested in your directory
3. **Set up the ArcGIS Pro Python environment:**
   The 01b Extract Raster to Polygon tool in the wtw-utils toolbox uses the exactextract Python package, which is not included in ArcGIS Pro's default `arcgispro-py3` environment. Each user needs to set up a cloned environment with exactextract installed, once per ArcGIS Pro installation:
   - Close ArcGIS Pro.
   - From the Start menu, open **ArcGIS → Python Command Prompt**.
   - Clone the default environment: `conda create --clone arcgispro-py3 --name wtw-data-prep`. This can take several minutes and goes quiet near the end; wait for `Executing transaction: done` before continuing. A clone can need 20 GB or more of free space, depending on what the default environment contains (e.g. the deep learning packages).
   - Activate the new environment: `activate wtw-data-prep`
   - Install exactextract: `pip install exactextract --no-deps`. pip installs the exactextract build that matches your ArcGIS Pro Python version. Use pip with `--no-deps` rather than `conda install`, which can replace ArcGIS Pro's own packages and break arcpy.
   - Open ArcGIS Pro, go to **Project → Package Manager**, set **Active Environment** to `wtw-data-prep`, and restart ArcGIS Pro.
   - When 01b runs, its first message shows the exactextract version and location in use, e.g. `Using exactextract 0.3.0 (C:\Users\...\envs\wtw-data-prep\...)`. If exactextract cannot be loaded, the tool reports the active environment and Python version.
   - After upgrading ArcGIS Pro, the cloned environment still contains the previous version's packages and may stop working. Delete it by running `conda env remove --name wtw-data-prep` in the Python Command Prompt, then repeat these steps.

- **Alternative for limited disk space (e.g. AVD):** on AVD, conda environments are stored on a profile disk of about 30 GB, which a clone can fill. Instead of cloning, install exactextract into your Python user folder, which needs a few MB and no admin rights:
    - In the Python Command Prompt, leave the default `arcgispro-py3` environment active and run: `python -m pip install --user exactextract --no-deps`
    - Open ArcGIS Pro. No environment change is needed. When 01b runs, its first message should show a location under `AppData\Roaming\Python\Python3xx`.
    - The user folder is shared by every Python of that version, so don't `pip install --user` other packages such as numpy or GDAL; a second copy can break arcpy. If a cloned environment with exactextract is active, its copy is used first.
    - After upgrading ArcGIS Pro to a version with a new Python (e.g. 3.13 to 3.14), run the pip command again.
    - If an earlier clone failed partway (e.g. `No space left on device`), delete the `wtw-data-prep` folder in `%LOCALAPPDATA%\ESRI\conda\envs` and run `conda clean --all` to recover the space.

Data Prep Folder Structure and Run Steps

The following outlines the required folder structure and run steps for building a WTW project. This process involves creating the WTW project folder structure, copying your AOI shapefile, the wtw-data-prep2/R folder, and the `setup.toml` file into the project directory, creating an RStudio project file, editing the `setup.toml`, and finally running the `__pipeline__.R` script.

1. **Create WTW project folder structure**
   Each WTW project should follow a consistent folder structure. This structure helps keep your prep work organized and aligned with NCC conventions.
    - Create base directory for WTW projects e.g. `C:/PRZ/WTW`
    - Create a subfolder for your region, e.g. `C:/PRZ/WTW/REG_ON`
    - Within the regional folder, create your WTW project directory for your specific area of interest e.g. `C:/PRZ/WTW/REG_ON/ELOC`

2. **Copy `AOI.shp`, R folder and `setup.toml` to WTW project directory**
    - After creating your WTW project directory, create a subfolder named aoi: *e.g.* `C:/PRZ/WTW/REG_ON/ELOC/aoi`
    - In the aoi folder, add a shapefile that defines your area of interest.
        - The `aoi.shp` will be projected to *Canada_Albers_WGS_1984* in the national pipeline.
        - It is recommended to dissolve the boundary, so the AOI is a single, flat polygon.
    - Copy *wtw-data-prep2/R* from your repository directory to the WTW project directory**:** *e.g.* `C:/PRZ/WTW/REG_ON/ELOC/R`
    - Copy *wtw-data-prep2/*`setup.toml` from your repository directory to your project directory root folder

3. **Create RStudio Project (`.Rproj`)**
    - Open a new RStudio session.
    - Go to File → New Project → Existing Directory.

- Navigate to your WTW project folder and click Create Project.

This will initialize an RStudio project and create a `.Rproj` file in your WTW project directory. This file should be used to open your project going forward, as it automatically sets the working directory and ensures all file paths are relative to your project.

4. **Edit** `setup.toml` **file**
   Open your WTW project by launching RStudio from the project's `.Rproj` file. Then, navigate to the `setup.toml` file and open it for editing. Update the `[local]` and `[wtw]` sections to reflect the structure and details of your specific WTW project. These parameters are programmatically accessed and used in the `__pipeline__.R` script.

   `[local]`

   - `project_dir` = project directory, the full path to your WTW project directory
     e.g. `C:/PRZ/WTW/REG_ON/ELOC`

   - **`natdata_dir`** = natdata directory, full path to your national data bundle
     e.g. `C:/PRZ/NAT_DATA/NAT_DATA_20250801`

   - **`aoi_shp`** = name of the shapefile located in your `/aoi` folder, including the `.shp` extension
     e.g. `Eloc_Boundary.shp`

   - `punits` = relative file path to your planning units.
     e.g. `/aoi/pu_1km.tif`

   - `wtw_metadata` = relative path to your `wtw-metadata.csv`
     e.g. `/wtw/metadata/wtw-metadata.csv`

   `[wtw]`

   - `project_name` = name of your project. It's recommended to prefix with the region and suffix with scale. When your WTW project is copied to the server, it becomes available as a built-in project. Following this naming format helps keep projects organized and makes it easier to search for those matching a specific region and scale.
     e.g. REG ON: ELOC 1km

   - `file_name` **=** A lowercase prefix used for the four WTW output files. Keeping the name all lowercase is recommended for consistency.
     e.g. eloc

   - `author` = name of the person preparing the data.

   - `email` = email address of the person preparing the data.

   - `groups` = set to either "public" or "private". If set to private, and the project is published on the Sites server, access will be restricted to NCC staff only.

5. **Run `__pipeline__.R`**
   The `__pipeline__.R` script is the main entry point for building a WTW project. It sequentially runs all required steps, including:

   - `00_setup.R`: sets paths based on the `setup.toml` file and installs any required packages.
   - `01_init_project_folder.R`: creates the output WTW project folder structure.
   - `02_aoi_to_1km_grid.R`: builds a 1km grid from the AOI shapefile.
   - **`03_natdata.R`**: extracts national 1km data that intersects your AOI.
   - `04_metadata.R`: creates the `wtw-metadata.csv` file that records information for each layer.
   - `05_wtw.R`: translates format-ready spatial data and metadata into the four WTW files.

   In RStudio, open the **`__pipeline__.R`** script. Select all the code using **Ctrl + A**, then run it by either clicking the **Run** button in the top right of the script editor or pressing **Ctrl + Enter**.

   When running `__pipeline__.R` for the first time, required R packages will be automatically installed into your system library. This initial setup may take some time due to package downloads. On subsequent runs, the pipeline will reuse the installed packages and skip reinstallation.

6. **Validate outputs**
   Once `__pipeline__.R` completes, navigate to your project folder to view the structure and output files.
   - `/aoi`
     - `aoi.shp`: reprojected AOI boundary
     - `pu_1km.shp`: vector planning units
     - `pu_1km.tif`: raster planning units
     - `pu_1km_align.tif`: intermediate output not further used
   - `/dataprep`
     - `dataprep.csv` designed to record regional variables for wtw-utils batch processing
   - `/dataprep/national`
     - empty folder designed to store inactive national data
   - `/dataprep/regional`
     - empty folder designed as a workspace for prepping regional data
   - `/dataprep/tmp`
     - empty folder designed to hold intermediate data to be deleted later
   - `/tifs`
     - excludes
     - includes
     - themes
       - habitat
       - species
         - eccc_ch, eccc_sar, iucn_amph, iucn_bird, iucn_mamm, iucn_rept, ncc_sar_aoh, ncc_sdm, nsc_end, nsc_sar, nsc_spp
     - weights

        - carbon, climate, connectivity, eservices, hotspots, pressures
- `/wtw`
    - `configuration.yaml`
    - `spatial.tif`
    - `attribute.csv.gz`
    - `boundary.csv.gz`
- `/wtw/metadata`
    - `wtw-metadata.csv`
- `/wtw/runs`
    - empty folder designed to store WTW runs

wtw-metadata.csv

Created in the `R/04_metadata.R` step, the `wtw-metadata.csv` file (in your project's `/wtw/metadata` directory) records information for each format-ready layer required by `R/05_wtw.R`. To add regional data to a national project, insert new rows with the required details below:

- **Type**
  Classify the format-ready TIF as either a "theme", "weight", "include", or "exclude".
- **Theme**
  When the Type is "theme", enter a theme name to group related layers. All data in a theme group must have the same units. Leave blank if Type is not a "theme".
- **File**
  Provide the file name including the `.tif` extension of the layer.
- **Name**
  Provide a display name for the layer.
- **Legend**
  Indicate a legend type based on the layer data. Choose "manual" for binary or categorical data, and "continuous" for continuous data.
- **Values**
  When Legend is "manual", provide the exact categorical values, separated by a comma and a space (e.g. 0, 1). Leave blank if legend is continuous.
- **Color**
  When Legend is "manual", provide hex color codes corresponding to the cell values, separated by a comma and a space (e.g., #00000000, #b3de69). The number of colors must match the number of Values.
  When Legend is "continuous" provide a name of the color ramp. Supported ramps can be viewed at wheretowork::color_palette() under Details.
- **Labels**
  When Legend is "manual" provide the labels corresponding to the cell values, separated by a comma and a space (e.g. absence, presence). The number of labels must match the number of Values.
- **Unit**
  Provide a unit for the layer (e.g. ha).
- **Provenance**
  Indicate if the layer is "regional" or "national".

- **Visible**
  Set Visible to TRUE if you want the layer to be displayed on the map when the app initially loads. Otherwise, set it as FALSE. It is recommended to set only a few layers to TRUE, as this can increase app load times. Once the app is loaded, layers can still be made visible dynamically.
- **Hidden**
  Set Hidden to TRUE to prevent the layer from being displayed on the map. The layer remains available for optimization but cannot be viewed. This setting is designed to protect sensitive layers. Otherwise, set it as FALSE.
- **Downloadable**
  Set Downloadable to TRUE to allow the layer to be downloaded. For sensitive layers, set it to FALSE to prevent downloads.
- **Goal**
  When Type is "theme", provide a numeric goal value between 0 and 1. This value is converted to a percentage in Where To Work and sets the initial goals when the app loads. Goals can still be adjusted dynamically after the app has loaded.

Removing Data

Some layers may be unnecessary in a project following stakeholder review. Removing them reduces the size of the four generated WTW files.

To remove data, move the unneeded format-ready TIFs from your project's `/tifs` directory into the appropriate staging folder. These folders are designed to temporarily hold data before activation into the `/tifs` directory:

- `/dataprep/national`
- `/dataprep/regional`

After relocating the selected TIFs, update `wtw-metadata.csv` so it matches the contents of the `/tifs` directory.

1. **Create a backup:** Make a copy of `wtw-metadata.csv` and append today's date in YYYYMMDD format (e.g., `wtw/metadata/wtw-metadata_20250829.csv`). This versioning ensures you can roll back if needed.

2. **Update the production file:** In the active `wtw-metadata.csv` (the production-ready sheet), delete the rows corresponding to the relocated TIFs.

3. **Regenerate outputs:** Re-run the **# 05 Build WTW project** code chunk in `__pipeline__.R` to rebuild the four WTW files.

In practice, most reasons for removing data are to reduce overall project size. However, note that in WTW, you can also simply untoggle layers you do not want included in the optimization.

Adding Regional Data

There are two scenarios for preparing regional data:

a) adding it to an existing **NAT_1KM** project

b) adding it to a **Custom Grid**.

In both cases, the workflow is the same. The steps include summarizing conversion-ready GIS layers to the vector planning units, generating format-ready TIFs, updating the `wtw-metadata.csv`, and running the `R/05_wtw.R` formatting script in `__pipeline__.R`. To support this process, a custom ArcGIS Toolbox (wtw-utils) has been developed.

1. **Summarize conversion-ready regional GIS layers to the vector planning units.**
   This involves intersecting vector layers and summarizing their geometry, running zonal statistics on continuous raster data, and calculating area coverage for categorical rasters. The resulting values are stored in the planning units' attribute table, making them ready for conversion into format-ready TIFs.

2. **Generate format-ready TIFs**
   Each extracted column from the vector planning units attribute table is rasterized into a TIF and saved in the project's tifs directory. All TIFs in this folder must share the same projection, extent, resolution, number of rows and number of columns.

3. **Edit the** `wtw-metadata.csv`
   Each TIF in the project's tifs directory must be documented in the `wtw-metadata.csv`. Ensure that each entry is complete, accurate, and corresponds exactly to the required WTW information.

4. **Run # 05 Build WTW project code chunk in** `__pipeline__.R`**.**
   The `R/05_wtw.R` script uses the format-ready TIFs in the project's tifs directory together with the `wtw-metadata.csv` as inputs, and generates the four WTW input files.

   In `__pipeline__.R`, highlight the code starting from `source("R/05_wtw.R")` through to the closing bracket of the `build_wtw_project` function. With the code highlighted, you can either click the **Run** button at the top right of the RStudio session or press **Ctrl + Enter**.

   If you are in a new RStudio session, you must first run the **Setup** code chunk to load all `setup.toml` paths into memory. To do this, highlight the code from `source("R/00_setup.R")` to `wtw <- configs$wtw` and run the block.

   When executed, the `build_wtw_project` function will recursively scan your project's `/tifs` folder, list all TIF files, and use `wtw-metadata.csv` to associate each file with its corresponding metadata. **It is important that every TIF file has a matching metadata row; if there is not a one-to-one match, the tool will fail.**

ArcGIS custom toolbox (wtw-utils)

wtw-utils is a custom ArcGIS Pro toolbox that structures spatial datasets into a format compatible with WTW projects. It streamlines regional data preparation and supports configurable batch processing.

- Toolbox location: `wtw-utils.atbx` in your `wtw-data-prep2` repository, e.g. `C:/Github/wtw-data-prep2/wtw-utils.atbx`
- Supporting Python scripts: the `wtw-utils` folder in your repository, e.g. `C:/Github/wtw-data-prep2/wtw-utils`
- Python environment: a cloned ArcGIS Pro environment with exactextract installed, e.g. `wtw-data-prep`, or exactextract in your Python user folder where disk space is limited (see Initial setup). Required by 01b Extract Raster to Polygon.

The script tools extract conversion-ready regional data into the vector planning unit attribute table:

- **Vector data** → area, length, or count (by geometry)
- **Continuous rasters** → zonal statistics
- **Categorical rasters** → area summaries

These outputs are rasterized into format-ready TIFs for use in the `R/05_wtw` step in `__pipeline__.R`.

Script Tools

Detailed descriptions for each tool and its input parameters are available in the helper script tool metadata, viewable directly within ArcGIS Pro.

- **00 Create Custom Grid**
  Generates a user-defined equal-area square grid with both vector and raster outputs that serve as planning units. Use this tool only if a grid size or projection different from the National 1 km standard is required to meet regional needs.
- **01a Extract Vector to Polygon**
  Intersects the input vector data with the polygon and summarizes area, length, or count based on the geometry type. The results are then joined to the input polygon's attribute table using the dynamically generated `WTWID` field.
- **01b Extract Raster to Polygon**
  Extracts zonal statistics from continuous rasters or area coverages from categorical rasters and joins the results to the input polygon's attribute table. Input rasters can remain in their source projection and cell size; the polygon is reprojected as needed to ensure alignment. A `WTWID` field is dynamically created on the input polygon to support the join. Categorical rasters do not require reclassification, as this is handled internally.

  The tool uses the **exactextract** Python package, which accounts for partially covered grid cells when calculating statistics or coverage. It must be installed in ArcGIS Pro's active Python environment or your Python user folder (see Initial setup).
- **02 Rasterize Columns**
  Converts selected columns from a polygon attribute table into format-ready TIFs.

01a, 01b and 02 each have an **Input mode** setting, and the tool shows only the inputs for the chosen mode:

- **Single layer** (01a, 01b): one layer, with its unit or statistic and a short name.
- **Multiple layers** (01a, 01b) or **Multiple fields** (02): an editable table. 01a and 01b fill it from a geodatabase or folder; 02 fills it from the input polygon's numeric fields. Rows can be added, removed or edited before running.
- **Batch CSV file** (01a, 01b, 02): the rows of a dataprep CSV (see Dataprep.csv below). Recommended for regional projects, as the CSV records every input.

Inputs are checked before the tool runs, for example short names, units that don't fit a layer's geometry, missing cell values for area, and TIF names. Problems are reported by table row or CSV line. Only the inputs for the chosen mode are checked; values left in the other modes' inputs are ignored.

Dataprep.csv

Regional data preparation can utilise a `dataprep.csv` file to batch process files. Running `__pipeline__.R` will generate a template `dataprep.csv` in the `/dataprep` folder. The 01a, 01b and 02 tools can then read it in their Batch CSV file mode.

The `dataprep.csv` file supports batch processing of multiple vector and raster inputs by storing the required parameters for several ArcGIS script tools in a single sheet. In addition, several recommended columns are included to support better documentation.

Columns and data (dataprep.csv)

- `conversion_ready_input` **(required)**:
  Full path to the spatial dataset, including file name and extension. The dataset must be cleaned and conversion-ready. For vectors, this means free of bad geometry and overlap errors. For rasters, pixels must be vetted. The analyst is responsible for ensuring these standards are met. This parameter is used by the 01a Extract Vector to Polygon and 01b Extract Raster to Polygon tools.
  e.g. `C:/Github/wtw-data-prep2/tests/data/vector.gdb/waterbody_on`

- `short_name` **(required):**
  A name for the vector or raster file, limited to **9 characters or fewer**. This is used to create a new field in the input polygon attribute table and must comply with shapefile field name constraints. Use letters, numbers and underscores only, and don't start with a number. This parameter is used by the 01a Extract Vector to Polygon and 01b Extract Raster to Polygon tools, and by 02 Rasterize Columns as the field to rasterize.
  e.g. *LAKE*

- `tif_output` **(required):**
  Full path for the output TIF, including file name. The `.tif` extension is added if left off, and the file name can use letters, numbers, underscores and hyphens. This defines where the

extracted regional data will be saved as a format-ready TIF and is required for the 02 Rasterize Columns tool. It is recommended to follow the standard naming convention:
  - Prefix the file name with **"T"**, **"W"**, **"I"**, or **"E"** depending on the variable type.
  - Append _**REG**_ for regional variables or _**NAT**_ for national variables
  - For themes, also include the **theme group suffix**.

  e.g. `T_REG_HABITAT_FOREST.tif`

- `unit` **(required only for vector data):**
  Specifies the unit for summarizing vector data. Supported options include "km2", "ha", "m2", for polygons, "m", "km" for lines, and "count" for point. Values must be entered exactly as shown, without the quotes, as if selecting from a pick list. Leave blank if the data is raster. This parameter is used by the 01a Extract Vector to Polygon tool, which flags a unit that doesn't fit the layer's geometry.

- `stat` **(required only for raster data):**
  Specifies the zonal statistic for a continuous raster. Supported options are "mean", "max", "min", "sum", "median", "var", "stdev", and "mode". For categorical rasters, enter "area". Values must be entered exactly as shown, without the quotes, as if selecting from a pick list. Leave blank if data is vector. This parameter is used by the 01b Extract Raster to Polygon tool.

- `cell_value` **(required only for raster categorical data):**
  Specify the numeric cell value in the categorical raster for which you want to extract the area**.** Required when stat is "area"; leave blank if the raster is continuous or the data is vector. This parameter is used by the 01b Extract Raster to Polygon tool.
  e.g. *7*

- `datatype` **(required):**
  Specify if data is either "vector" or "raster". Values must be entered exactly as shown, without the quotes, as if selecting from a pick list. This parameter is used by the Extract Vector to Polygon and Extract Raster to Polygon tools.

- `provenance` **(optional):**
  Specifies whether the data is "national" or "regional". Values must be entered exactly as shown, without quotes, as if selecting from a pick list. This parameter is not used by any tools but serves for documentation purposes.

- `source` **(optional):**
  Enter the full R: drive path to the source data, or provide the web address (URL) where the source data was obtained.

- `notes` **(optional):**
  Enter any notes or comments related to the layer.

NAT_DATA Bundle (S: drive)

NAT_DATA houses Canada-wide production datasets that have been harmonized to the Canada_Albers_WGS_1984 projection and aligned with NCC's national 1km grid. The data is structured for compatibility to support processing pipelines and apps.

Datasets may be added or removed based on guidance from CP&P and IT. Some layers follow an annual update cycle, depending on source data availability. The bundle is actively maintained to ensure compatibility with current applications and future workflows. New versions are created as needed.

Versioning

Versioned bundles of NAT_DATA are stored at `S:\CONS_TECH\PRZ\DATA\NAT_DATA\__VERSIONS__`. Each version is zipped and suffixed with a date in YYYYMMDD format. To access the most current data, copy the folder with the latest date suffix.

Updating (copy from S: drive)

When a new bundle is released, copy it from the S: drive to your local machine. Then update your WTW project by editing `natdata_dir` in `setup.toml` and re-running the pipeline.

If your project includes custom layers or a modified `wtw-metadata.csv`, make sure to copy these files to a temporary location outside the WTW project before re-running the pipeline, as it will overwrite existing files. You can then copy them back and make any necessary updates.

wtw-data-prep2

The wtw-data-prep2 repository contains the R and Python code needed to take a project from start to finish, along with a custom ArcGIS Toolbox to support regional data preparation.

The repository is actively maintained and regularly updated. To ensure you're working with the latest code, always keep your local repository in sync with the source of truth on the NCC-CNC GitHub account.

Updating (`git pull`)

For every new project, it's recommended to run a `git pull` on the main branch before beginning the project setup to ensure you're using the most current version.

In some cases, updates may be made during the course of a WTW project build. In these situations, run another `git pull` on the main branch, and make sure to copy the updated R folder and `setup.toml` file into your active project directory.

To `git pull` the main branch follow these steps:

- **Open a new terminal in RStudio**
  Go to **Tools → Terminal → New Terminal**. This will open a terminal pane at the bottom of RStudio.

- **Navigate to your local wtw-data-prep2 repository e.g.**
  `cd "C:/GitHub/wtw-data-prep2"`
- **Switch to the main branch** (if you're not already on it):
  `git checkout main`
- **Pull the latest updates from GitHub**:
  `git pull`

This ensures your local copy is up to date with the latest version of the code from the NCC-CNC GitHub repository.

Regional GIS Data Management

Regional data must be copied locally to run WTW data prep. To stay organized, it is recommended to follow a similar approach as we use for national data, where a “bundle” folder is managed on AVD. This allows multiple team members to copy the zipped folder locally, ensuring that everyone on the regional team is accessing the same data. The exact organization of regional data is left to each regional team to decide.